



Third Year Engineering

21BTCS501-Operating System

Class - T.Y. CORE 6 (SEM-I)

Unit - V

STORAGE MANAGEMENT

AY 2024-2025 SEM-I





Unit-V Syllabus

- | | |
|--------------------------------|---------------------------------|
| 1. File-System Interface, | 10. Directory Implementation, |
| 2. File Concept, | 11. Allocation Methods, |
| 3. Access Methods, | 12. Free-Space Management, |
| 4. Directory Structure, | 13. Efficiency and Performance, |
| 5. File-System Mounting, | 14. Recovery, |
| 6. File Sharing, Protection, | 15. Mass-Storage Structure, |
| 7. File-System Implementation, | 16. Disk Structure, |
| 8. File-System Structure, | 17. Disk Scheduling, |
| 9. File-System Implementation, | 18. Swap-Space Management |





- What is a file-system?
 - An abstract representation of physical devices capable of storing and retrieving binary information (files)
 - consists of partitions (volumes (NT), minidisks, etc), directories, and files
 - “virtual” file-systems may be smaller or greater in size than the physical devices used to store the information
- What is a file?
 - A file is an abstract data type, defined by the operations that can be performed on them
 - A named collection of related information recorded on a secondary storage (non-volatile) device
 - Uniform logical view of information storage
 - A logical storage unit (mapped by the OS onto a physical device)
- Who decides how the meaning is encoded (structure)?
 - Operating system
 - Program



- A file is a sequence of bits, bytes, lines or records whose format encodes meaning as defined by the file's creator or user:
 - “Freeform” structure: sequence of words, bytes
 - Simple regular structure: fixed or variable length records
 - Complex structures: formatted document, relocatable load file
- **File types:**
 - Data (numeric, character, binary)
 - Executable (loader can bring into memory and execute)
 - Directory (keeps information about other files)
- **Attributes:**
 - **Name** – only information kept in human-readable form.
 - **Type** – needed for systems that support different types.
 - **Location** – pointer to file location on device.
 - **Size** – current file size.
 - **Protection** – controls who can do reading, writing, executing.
 - **Time, date, and user identification** – data for protection, security, and usage monitoring.



File Attributes

- Name – only information kept in human-readable form
- Identifier – unique tag (number) identifies file within file system
- Type – needed for systems that support different types
- Location – pointer to file location on device
- Size – current file size
- Protection – controls who can do reading, writing, executing
- Time, date, and user identification – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk
- Many variations, including extended file attributes such as file checksum
- Information kept in the directory structure





File Operations

- Create
- Write – at **write pointer** location
- Read – at **read pointer** location
- Reposition within file - **seek**
- Delete
- Truncate
- **Open (F_i)** – search the directory structure on disk for entry F_i , and move the content of entry to memory
- **Close (F_i)** – move the content of entry F_i in memory to directory structure on disk



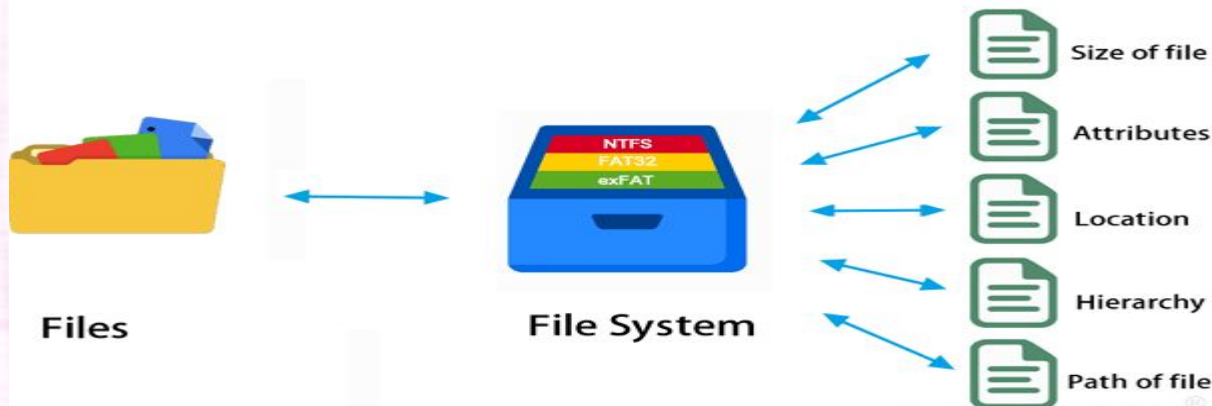
A file system is a method an operating system uses to store, organize, and manage files and directories on a storage device.

- ❖ **FAT** (File Allocation Table): An older file system used by older versions of Windows and other operating systems.
- ❖ **NTFS** (New Technology File System): A modern file system used by Windows. It supports features such as file and folder permissions, compression, and encryption.
- ❖ **ext** (Extended File System): A file system commonly used on Linux and Unix-based operating systems.
- ❖ **HFS** (Hierarchical File System): A file system used by macOS.
- ❖ **APFS** (Apple File System): A new file system introduced by Apple for their Macs and iOS devices.

A file is a collection of related information that is recorded on secondary storage.

Or file is a collection of logically related entities.

From the user's perspective, a file is the smallest allotment of logical secondary storage.





File Concept

- ❖ The operating system provides a uniform logical abstraction for the physical storage of information.
- ❖ Storage devices are nonvolatile.
- ❖ A file is a named collection of related information that is recorded on secondary storage.
- ❖ Contiguous logical address space

Types:

- ◆ Data ✓ numeric ✓ character ✓ binary
- ◆ Program ✓ Source, object and executable file formats





File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information





Open Files

- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files
 - File pointer: pointer to last read/write location, per process that has the file open
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - Disk location of the file: cache of data access information
 - Access rights: per-process access mode information





File Structure

- None - sequence of words, bytes
- Simple record structure
 - Lines
 - Fixed length
 - Variable length
- Complex Structures
 - Formatted document
 - Relocatable load file
- Can simulate last two with first method by inserting appropriate control characters
- Who decides:
 - Operating system
 - Program





Access Methods

- A file is fixed length **logical records**
- **Sequential Access**
- **Direct Access**
- **Other Access Methods**

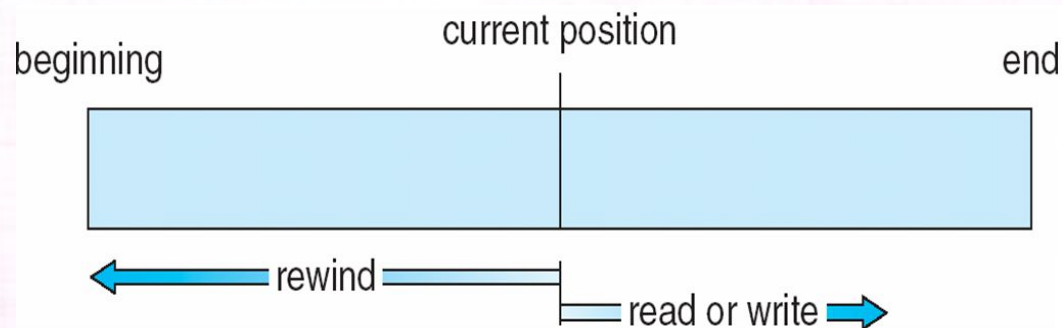




Sequential Access

- Operations
 - **read next**
 - **write next**
 - **Reset**
 - no read after last write (rewrite)

- Figure





Direct Access

- Operations
 - `read  $n$`
 - `write  $n$`
 - `position to  $n$`
 - `read next`
 - `write next`
 - `rewrite  $n$`

n = **relative block number**

- Relative block numbers allow OS to decide where file should be placed

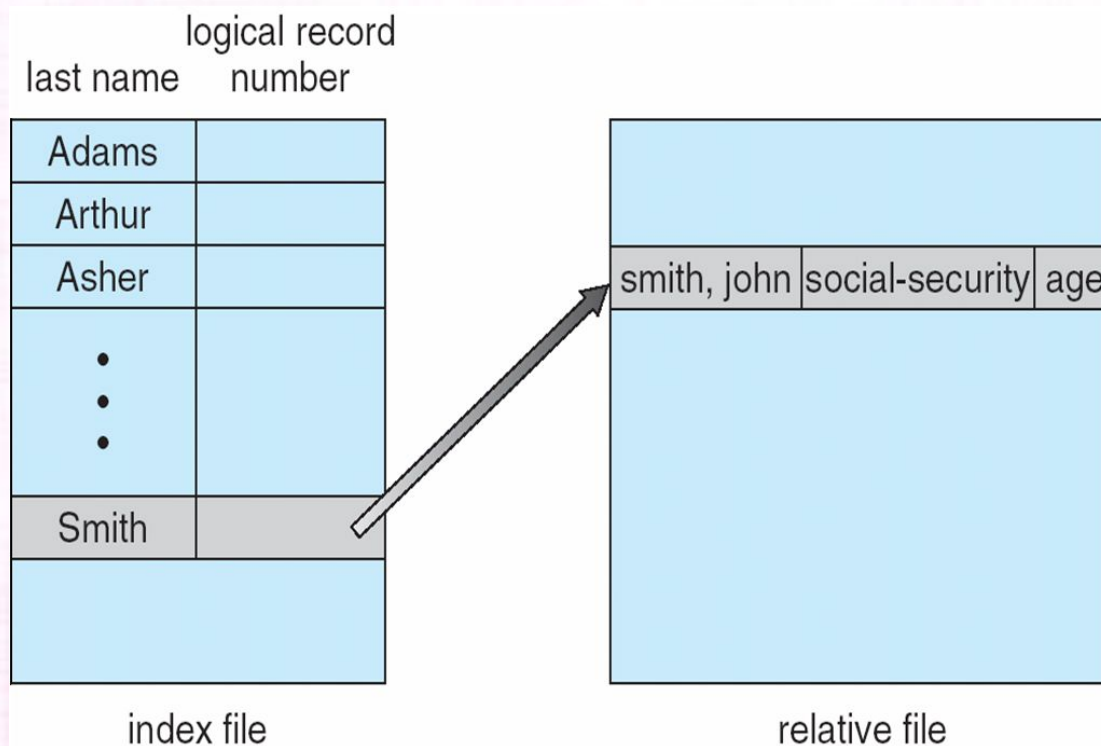


- Can be other access methods built on top of base methods
- General involve creation of an **index** for the file
- Keep index in memory for fast determination of location of data to be operated on (consider Universal Produce Code (UPC code) plus record of data about that item)
- If the index is too large, create an in-memory index, which an index of a disk index
- IBM indexed sequential-access method (ISAM)
 - Small master index, points to disk blocks of secondary index
 - File kept sorted on a defined key
 - All done by the OS
- VMS operating system provides index and relative files as another example (see next slide)





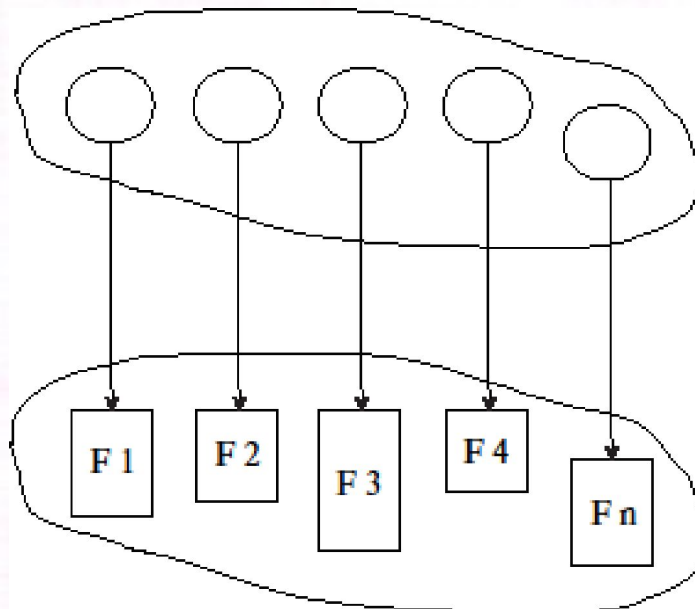
Example of Index and Relative Files





Directory Structure

- A collection of nodes containing information about all files



- Both the directory structure and the files reside on disk

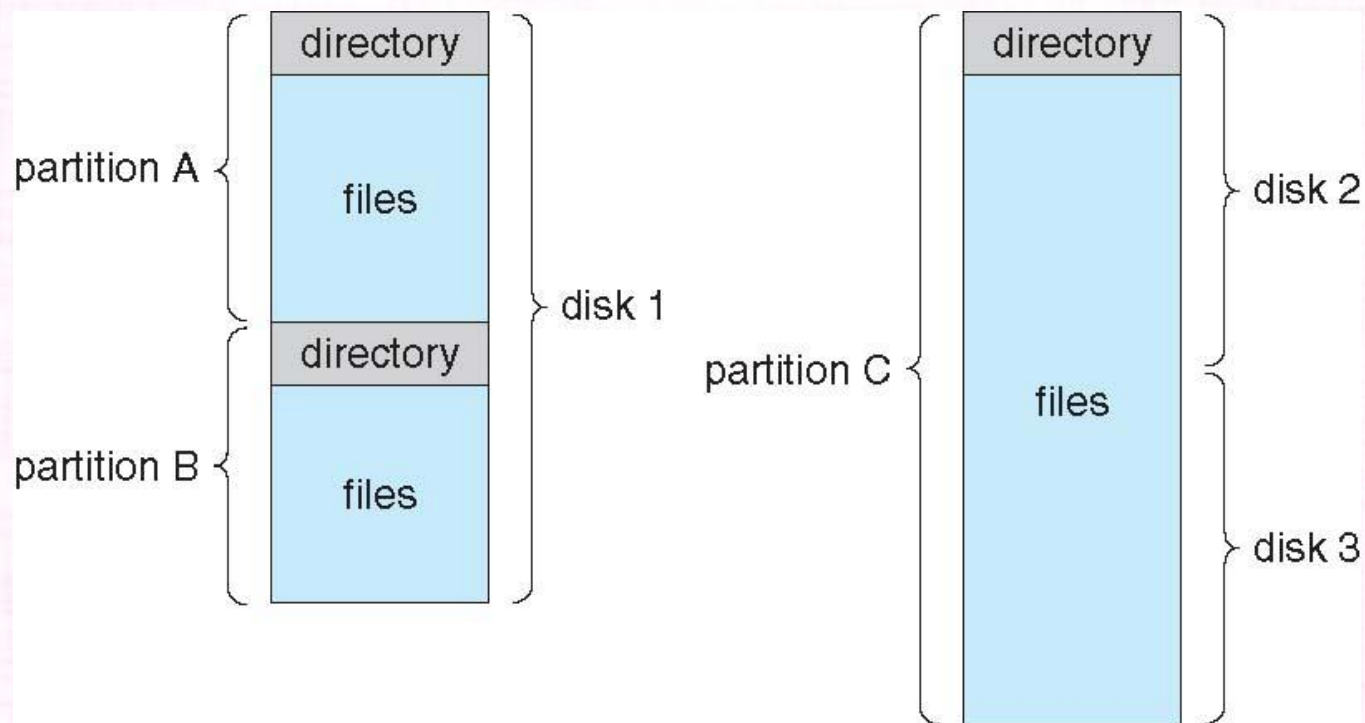


Disk Structure

- Disk can be subdivided into **partitions**
- Disks or partitions can be **RAID** protected against failure
- Disk or partition can be used **raw** – without a file system, or **formatted** with a file system
- Partitions also known as minidisks, slices
- Entity containing file system is known as a **volume**
- Each volume containing a file system also tracks that file system's info in **device directory** or **volume table of contents**
- In addition to **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer



A Typical File-system Organization





Types of File Systems

- We mostly talk of general-purpose file systems
- But systems frequently have many file systems, some general- and some special-purpose
- Consider Solaris has
 - tmpfs – memory-based volatile FS for fast, temporary I/O
 - objfs – interface into kernel memory to get kernel symbols for debugging
 - ctfs – contract file system for managing daemons
 - lofs – loopback file system allows one FS to be accessed in place of another
 - procfs – kernel interface to process structures
 - ufs, zfs – general purpose file systems





Operations Performed on Directory

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system





Directory Organization

The directory is organized logically to obtain

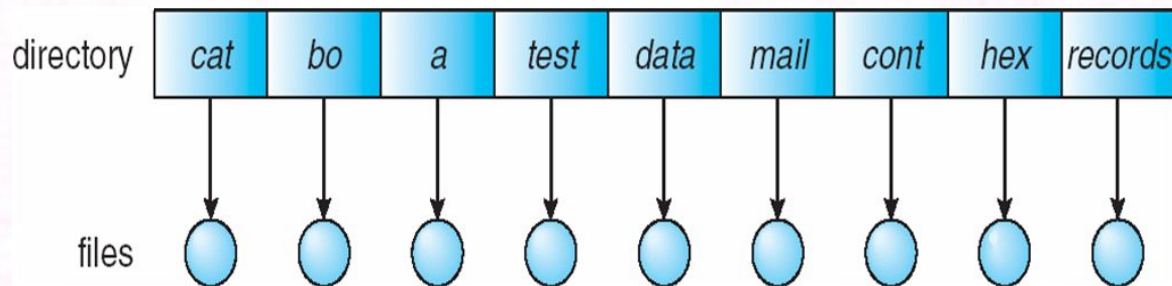
- Efficiency – locating a file quickly
- Naming – convenient to users
 - Two users can have same name for different files
 - The same file can have several different names
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, ...)





Single-Level Directory

- A single directory for all users



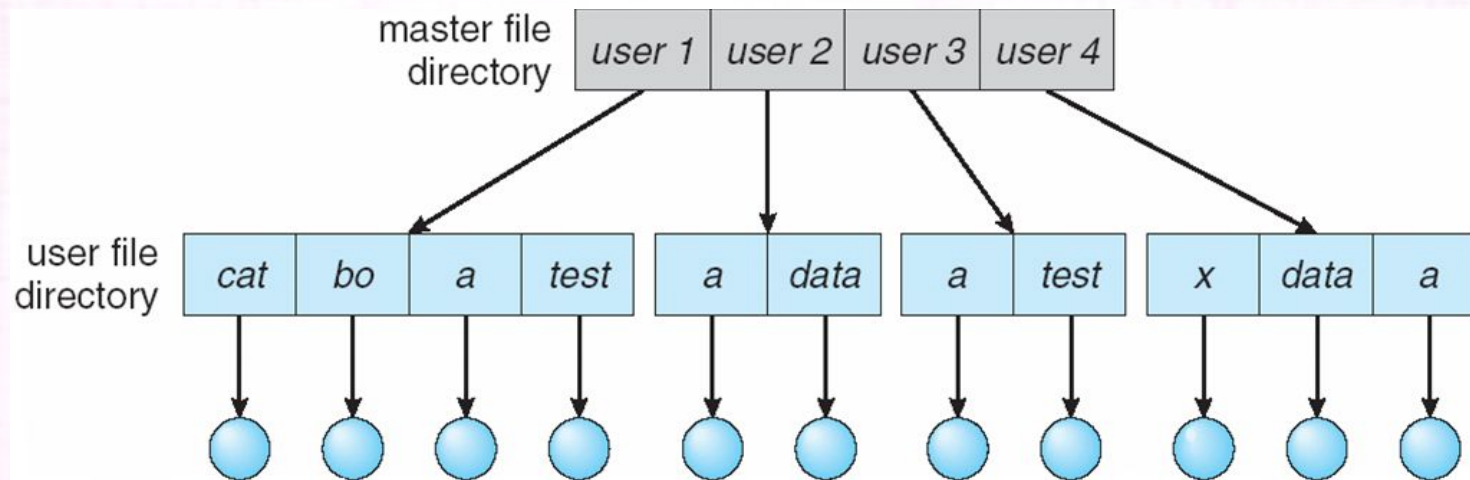
- Naming problem
- Grouping problem





Two-Level Directory

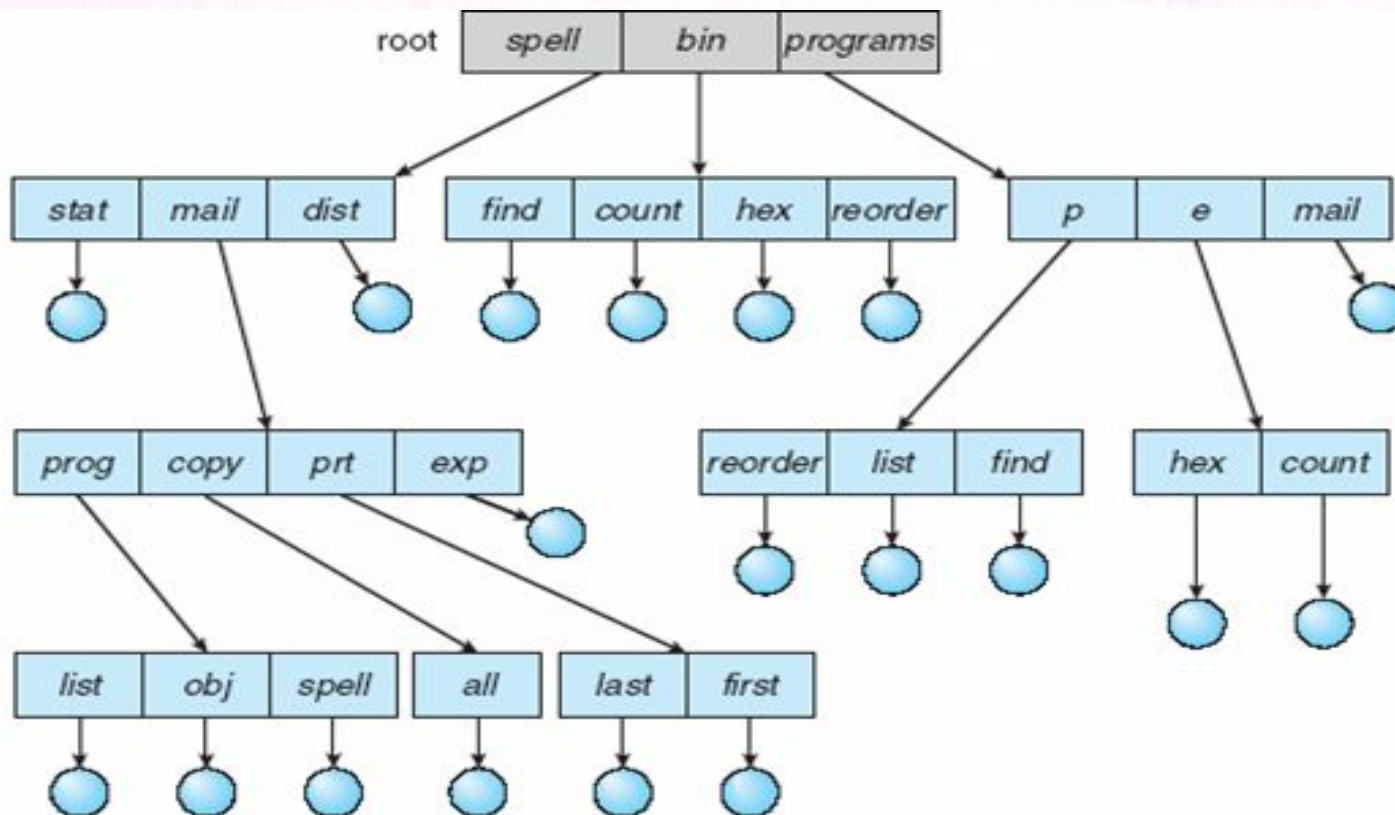
- Separate directory for each user



- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability



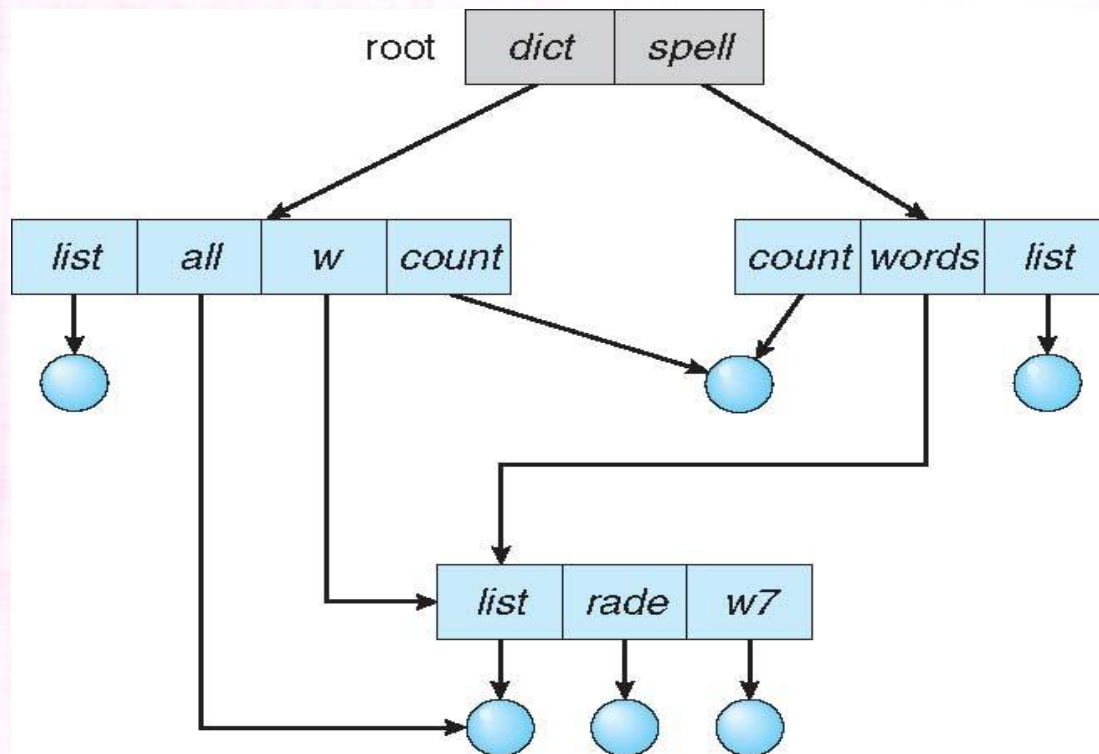
Tree-Structured Directories





Acyclic-Graph Directories

- Have shared subdirectories and files





Acyclic-Graph Directories (Cont.)

- Two different names (aliasing)
- If **dict** deletes **w/list** $\Rightarrow$ dangling pointer

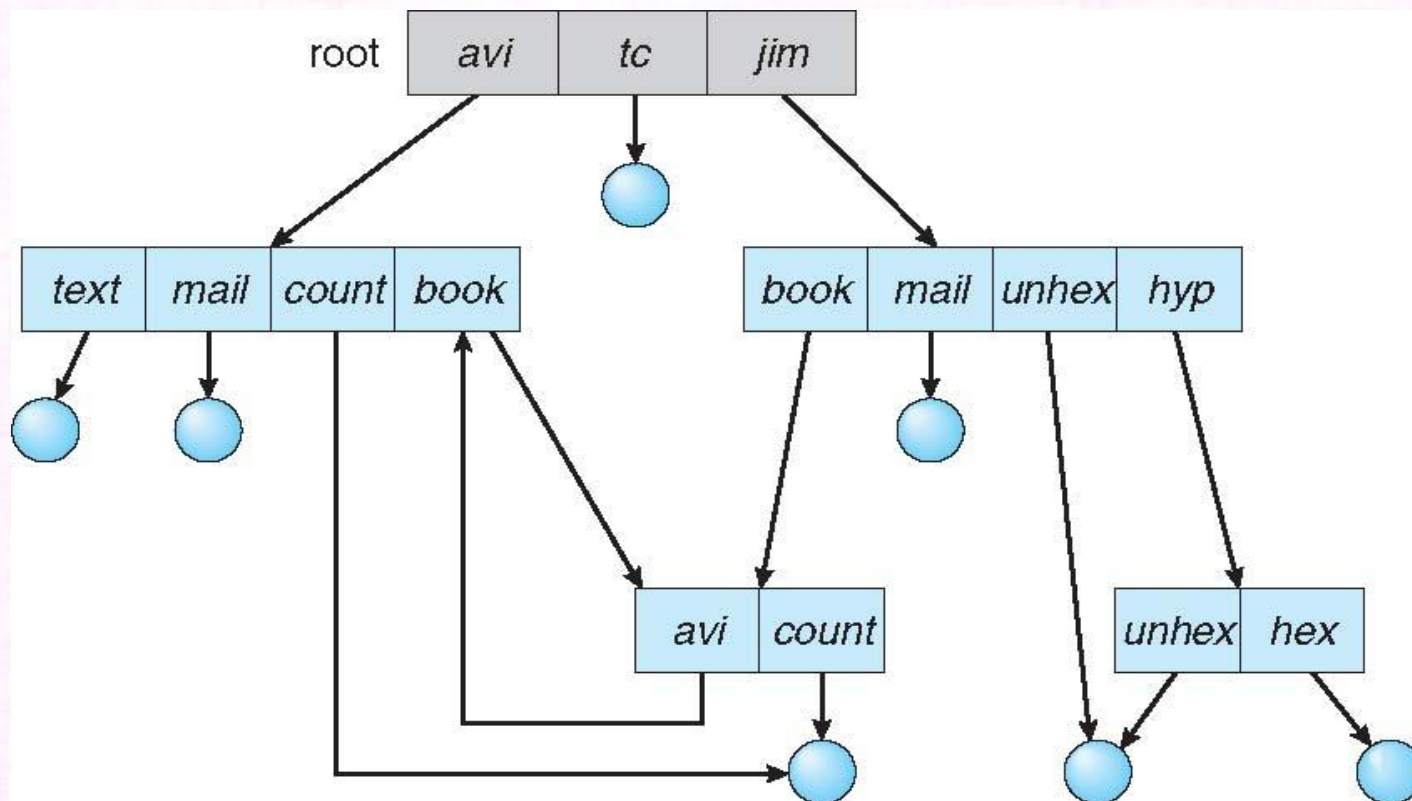
Solutions:

- Backpointers, so we can delete all pointers.
 - Variable size records a problem
- Backpointers using a daisy chain organization
- Entry-hold-count solution
- New directory entry type
 - **Link** – another name (pointer) to an existing file
 - **Resolve the link** – follow pointer to locate the file





General Graph Directory





General Graph Directory (Cont.)

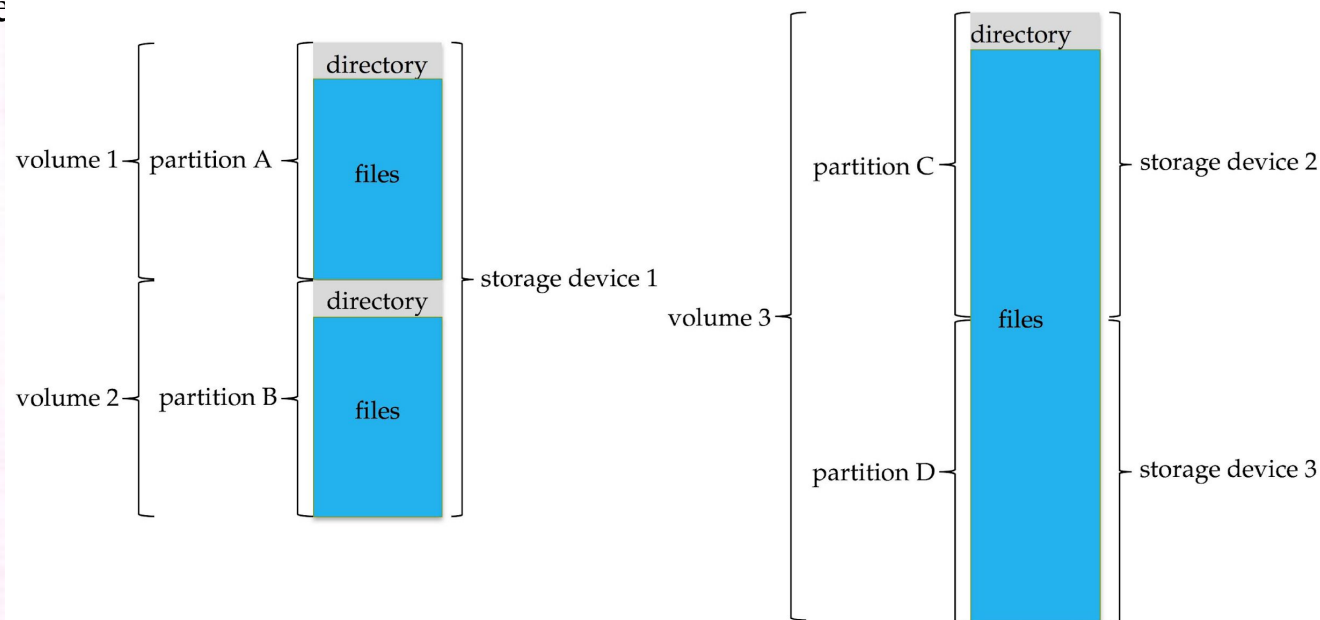
- How do we guarantee no cycles?
 - Allow only links to files not subdirectories
 - **Garbage collection**
 - Every time a new link is added use a cycle detection algorithm to determine whether it is OK





File System

- General-purpose computers can have multiple storage devices
- Devices can be sliced into partitions, which hold volumes
- Volumes can span multiple partitions
- Each volume usually formatted into a file system
- # of file systems varies, typically dozens available to choose from
- Typical storage device





Partitions and Mounting

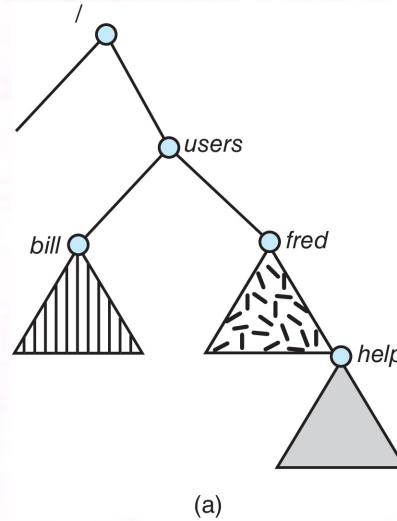
- Partition can be a volume containing a file system (“cooked”) or **raw** – just a sequence of blocks with no file system
- Boot block can point to boot volume or boot loader set of blocks that contain enough code to know how to load the kernel from the file system
 - Or a boot management program for multi-os booting
- **Root partition** contains the OS, other partitions can hold other OSes, other file systems, or be raw
 - Mounted at boot time
 - Other partitions can mount automatically or manually on **mount points** – location at which they can be accessed
- At mount time, file system consistency checked
 - Is all metadata correct?
 - If not, fix it, try again
 - If yes, add to mount table, allow access



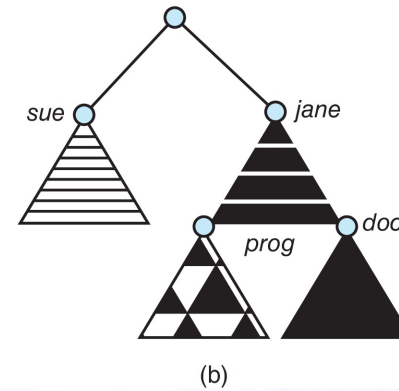


File Systems and Mounting

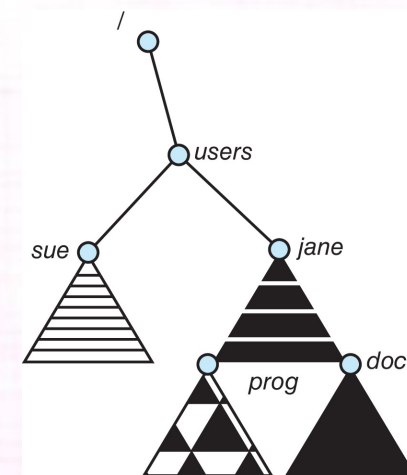
(a) Unix-like file system directory tree

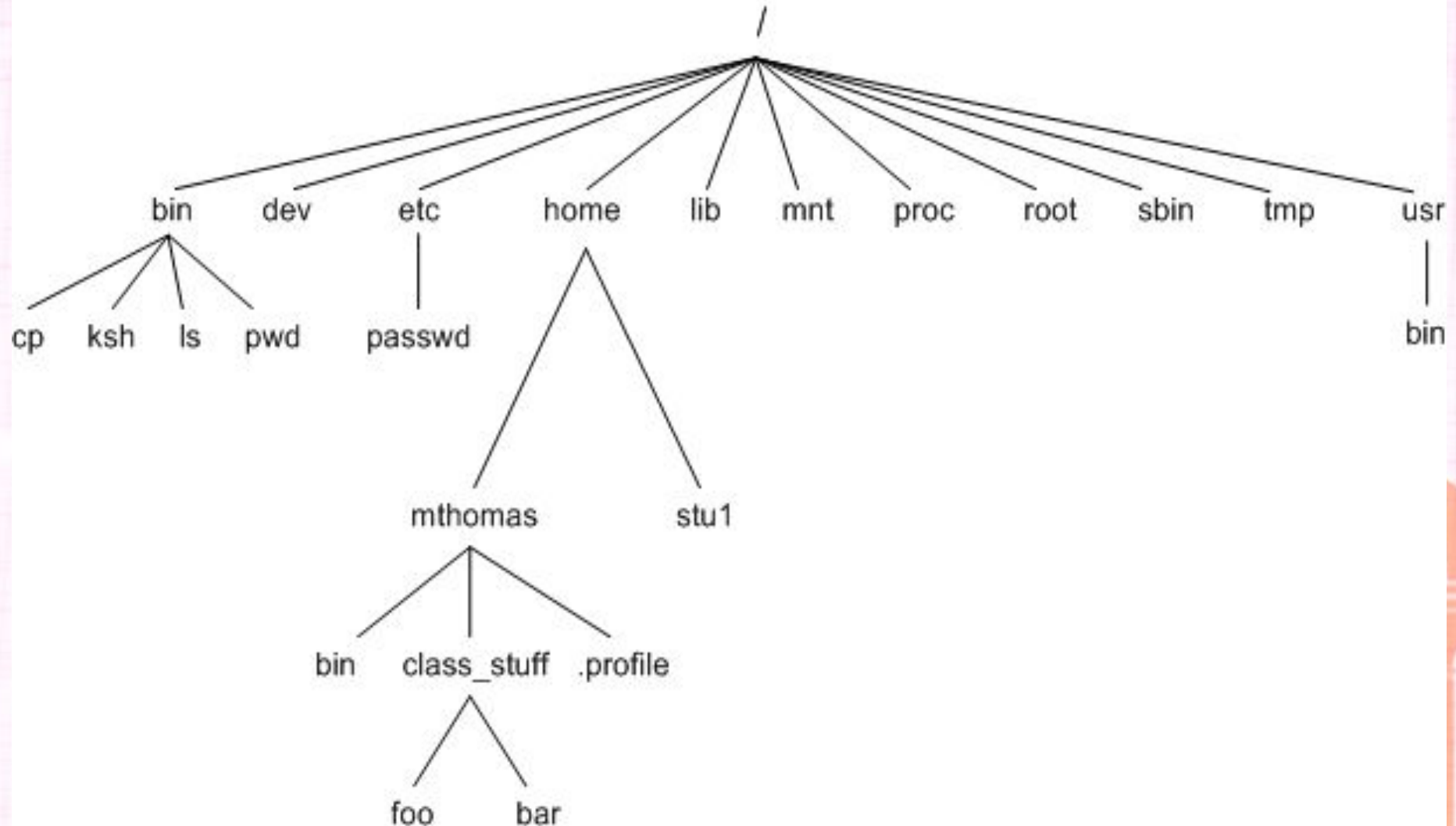


(b) Unmounted file system



After mounting (b) into the existing directory tree







File Sharing

- Allows multiple users / systems access to the same files
- Permissions / protection must be implemented and accurate
 - Most systems provide concepts of owner, group member
 - Must have a way to apply these between systems





Protection

- File owner/creator should be able to control:
 - What can be done
 - By whom
- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**





Access Lists and Groups in Unix

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

	RWX		
a) owner access	7	⇒	1 1 1
	RWX		
b) group access	6	⇒	1 1 0
	RWX		
c) public access	1	⇒	0 0 1



- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a file (say *game*) or subdirectory, define an appropriate access.

chgrp G game

- Attach a group to a file



A Sample UNIX Directory Listing

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/





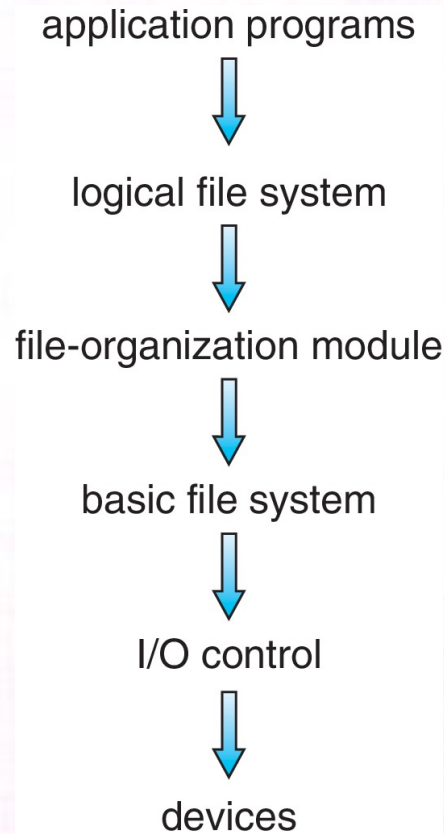
File-System Structure

- File structure
 - Logical storage unit
 - Collection of related information
- File system resides on secondary storage (disks)
 - Provided user interface to storage, mapping logical to physical
 - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- Disk provides in-place rewrite and random access
 - I/O transfers performed in blocks of sectors (usually 512 bytes)
- File control block (FCB) – storage structure consisting of information about a file
- Device driver controls the physical device
- File system organized into layers





Layered File System





File System Layers--- SELF Read

- Device drivers manage I/O devices at the I/O control layer
 - Given commands like
 - read drive1, cylinder 72, track 2, sector 10, into memory location 1060
 - Outputs low-level hardware specific commands to hardware controller
- Basic file system given command like “retrieve block 123” translates to device driver
- Also manages memory buffers and caches (allocation, freeing, replacement)
 - Buffers hold data in transit
 - Caches hold frequently used data
- File organization module understands files, logical address, and physical blocks
 - Translates logical block # to physical block #
 - Manages free space, disk allocation





File System Layers (Cont.)

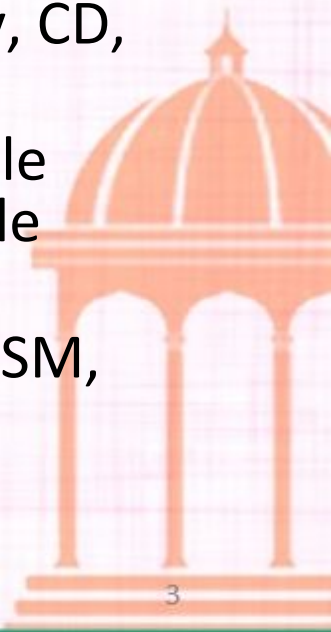
- Logical file system manages metadata information
 - Translates file name into file number, file handle, location by maintaining file control blocks (inodes in UNIX)
 - Directory management
 - Protection
- Layering useful for reducing complexity and redundancy, but adds overhead and can decrease performance
- Logical layers can be implemented by any coding method according to OS designer





File System Layers (Cont.)

- Many file systems, sometimes many within an operating system
 - Each with its own format:
 - CD-ROM is ISO 9660;
 - Unix has UFS, FFS;
 - Windows has FAT, FAT32, NTFS as well as floppy, CD, DVD Blu-ray,
 - Linux has more than 130 types, with extended file system ext3 and ext4 leading; plus distributed file systems, etc.)
 - New ones still arriving – ZFS, GoogleFS, Oracle ASM, FUSE





File-System Operations

- We have system calls at the API level, but how do we implement their functions?
 - On-disk and in-memory structures
- Boot control block contains info needed by system to boot OS from that volume
 - Needed if volume contains OS, usually first block of volume
- Volume control block (superblock, master file table) contains volume details
 - Total # of blocks, # of free blocks, block size, free block pointers or array
- Directory structure organizes the files
 - Names and inode numbers, master file table





File Control Block (FCB)

- OS maintains FCB per file, which contains many details about the file
 - Typically, inode number, permissions, size, dates
 - Example

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks





In-Memory File System Structures

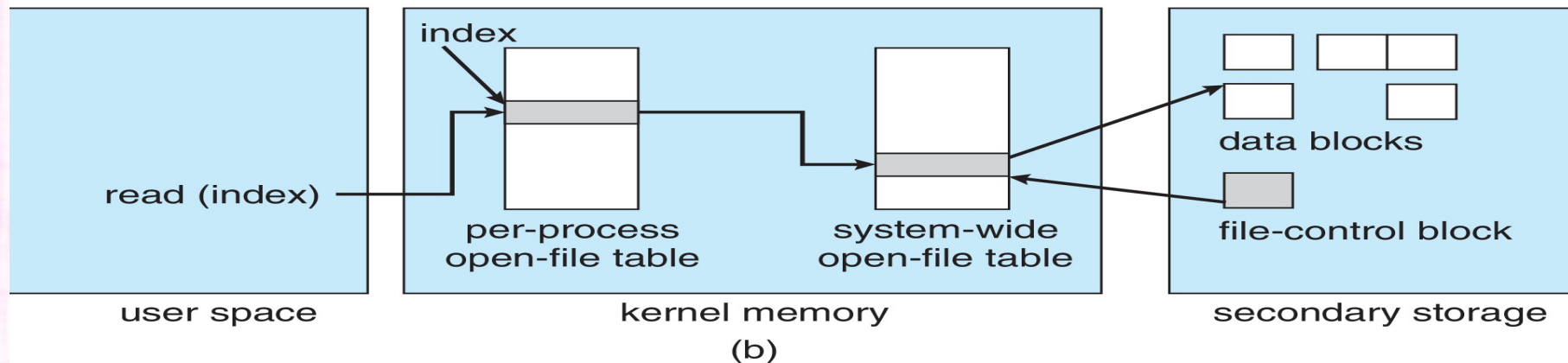
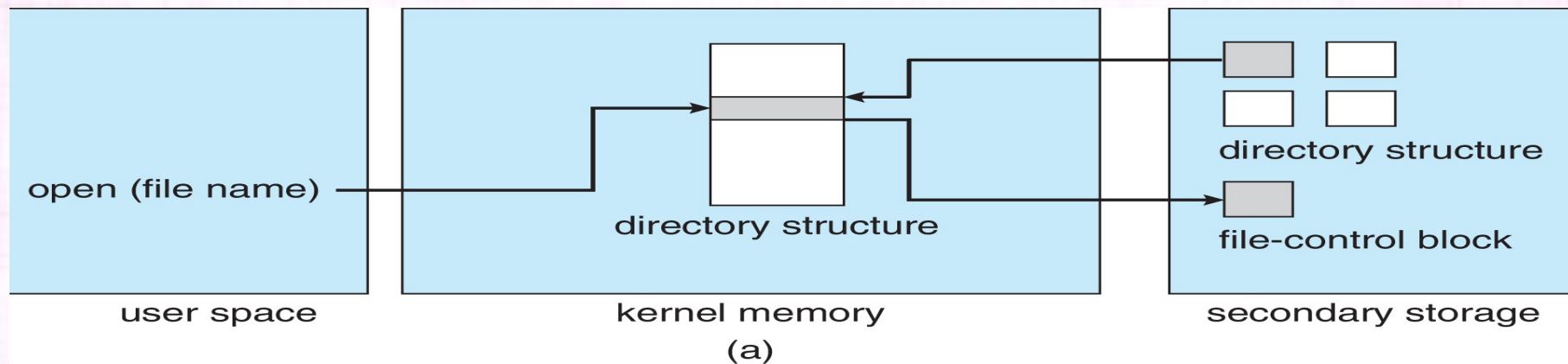
- Mount table storing file system mounts, mount points, file system types
- System-wide open-file table contains a copy of the FCB of each file and other info
- Per-process open-file table contains pointers to appropriate entries in system-wide open-file table as well as other info

Applications: Temporary Data Storage: Performance Optimization ; Caching

Limitations: Volatility; Ram Usage



In-Memory File System Structures (Cont.)



- Figure (a) refers to opening a file
- Figure (b) refers to reading a file



Directory Implementation

- Linear list of file names with pointer to the data blocks
 - Simple to program
 - Time-consuming to execute
 - Linear search time
 - Could keep ordered alphabetically via linked list or use B+ tree
- Hash Table – linear list with hash data structure
 - Decreases directory search time
 - Collisions – situations where two file names hash to the same location
 - Only good if entries are fixed size, or use chained-overflow method





Allocation Method

- An allocation method refers to how disk blocks are allocated for files:
 - Contiguous
 - Linked
 - File Allocation Table (FAT)





Contiguous Allocation Method

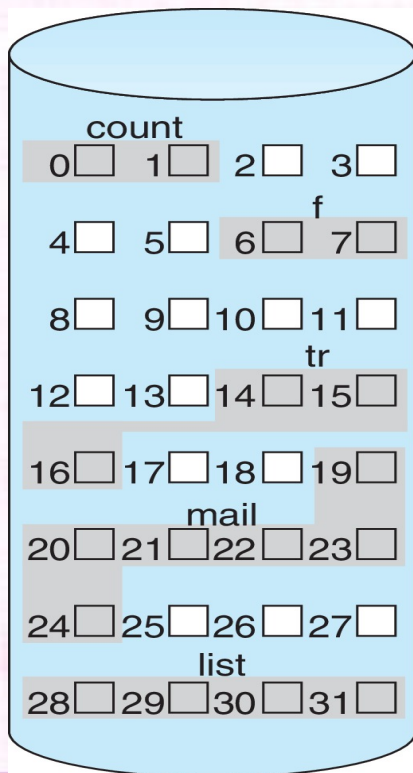
- An allocation method refers to how disk blocks are allocated for files:
- Each file occupies set of contiguous blocks
 - Best performance in most cases
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Problems include:
 - Finding space on the disk for a file,
 - Knowing file size,
 - External fragmentation, need for compaction off-line (downtime) or on-line





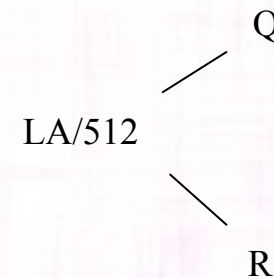
Contiguous Allocation (Cont.)

- Mapping from logical to physical (block size = 512 bytes)
- Block to be accessed = starting address + Q
- Displacement into block = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2





Advantages:

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b+k)$.
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

Disadvantages:

- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.





Linked Allocation

- Each file is a linked list of blocks
- File ends at nil pointer
- No external fragmentation
- Each block contains pointer to next block
- No compaction, external fragmentation
- Free space management system called when new block needed
- Improve efficiency by clustering blocks into groups but increases internal fragmentation
- Reliability can be a problem
- Locating a block can take many I/Os and disk seeks

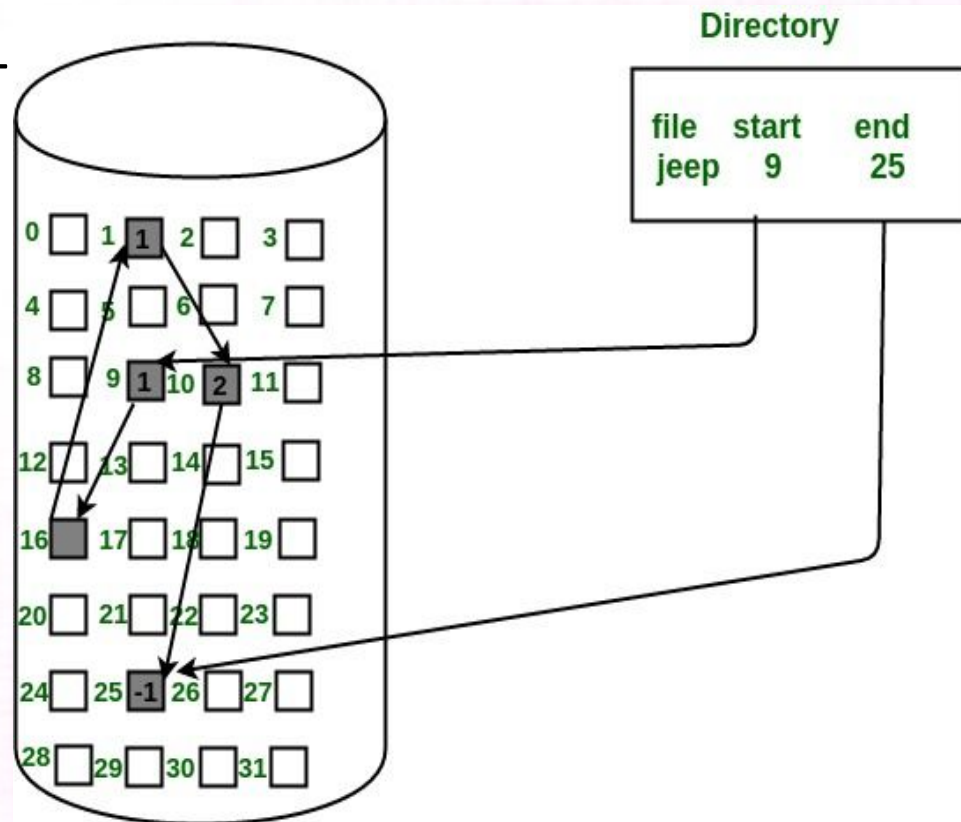




Linked Allocation (Cont.)

- Mapping
- Block to be accessed is the Qth block in the linked chain of blocks representing the file.
- Displacement into block = $R + 1$

LA/511 $\begin{cases} Q \\ R \end{cases}$





Advantages:

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages:

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.



Indexed Allocation

In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file.

Each file has its own index block.

The i th entry in the index block contains the disk address of the i th file block.

Advantages:

This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.

It overcomes the problem of external fragmentation.

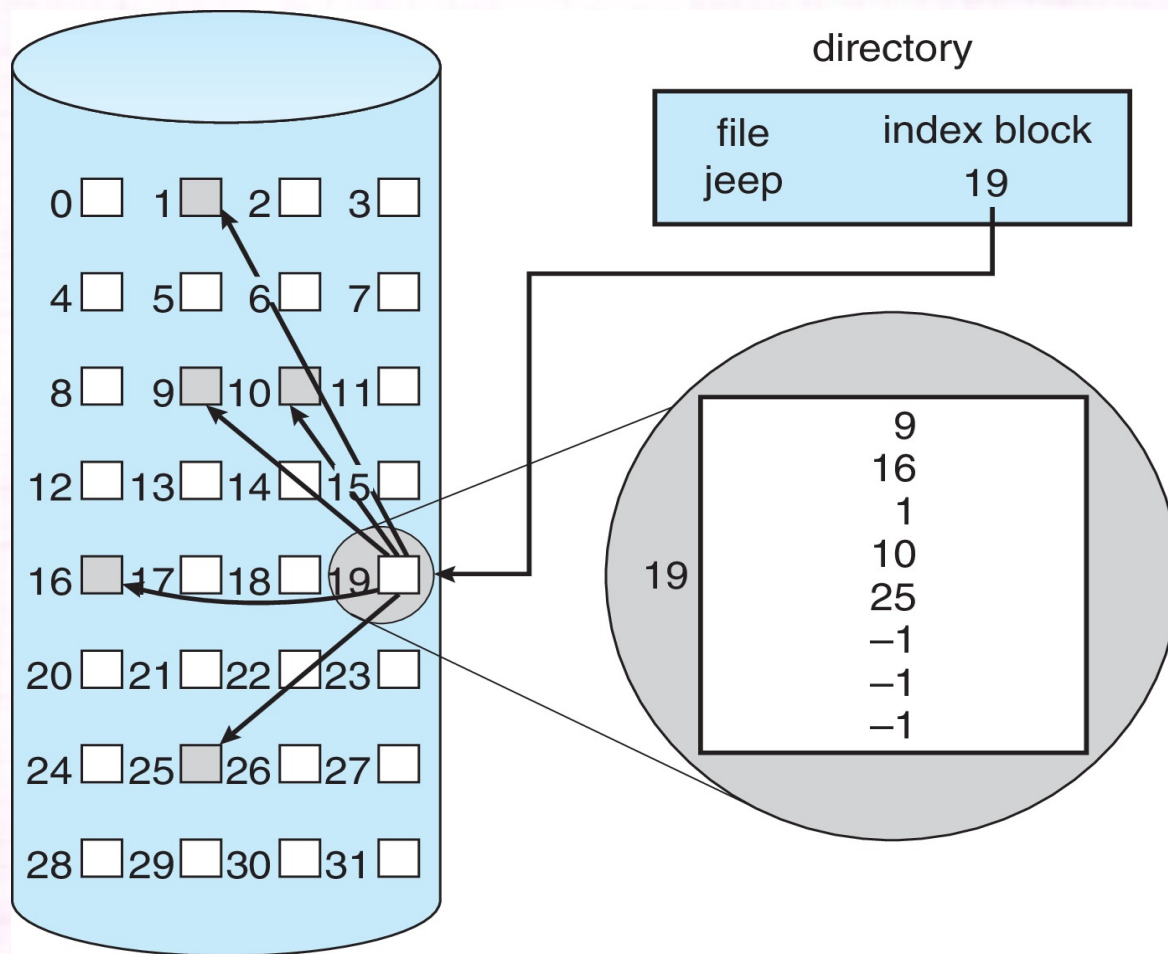
Disadvantages:

The pointer overhead for indexed allocation is greater than linked allocation.

For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.



Example of Indexed Allocation





FAT Allocation Method

- Beginning of volume has table, indexed by block number
- Much like a linked list, but faster on disk and cacheable
- New block allocation simple

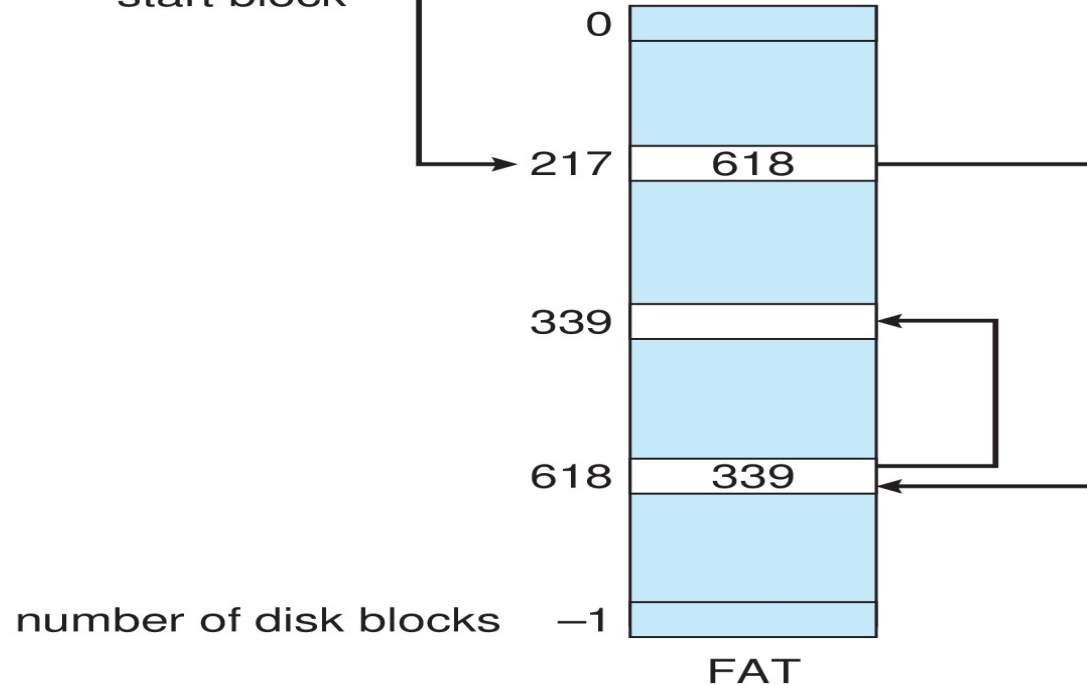




File-Allocation Table

directory entry

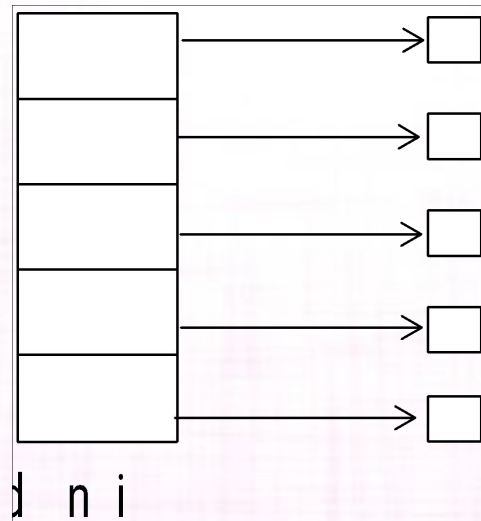
test	...	217
name		start block





Indexed Allocation Method

- Each file has its own index block(s) of pointers to its data blocks
- Logical view





Performance

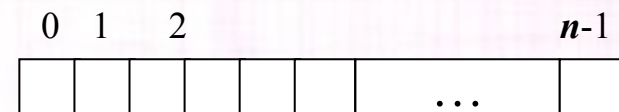
- Best method depends on file access type
 - Contiguous great for sequential and random
- Linked good for sequential, not random
- Declare access type at creation
 - Select either contiguous or linked
- Indexed more complex
 - Single block access could require 2 index block reads then data block read
 - Clustering can help improve throughput, reduce CPU overhead
- For NVM, no disk head so different algorithms and optimizations needed
 - Using old algorithm uses many CPU cycles trying to avoid non-existent head movement
 - Goal is to reduce CPU cycles and overall path needed for I/O





Free-Space Management

- File system maintains free-space list to track available blocks/clusters
 - (Using term “block” for simplicity)
- Bit vector or bit map (n blocks)



bit[*i*] ☐ 1 $\Rightarrow$ block[*i*] free
= ☐ 0 $\Rightarrow$ block[*i*] occupied

Block number calculation

(number of bits per word) *
(number of 0-value words) +
offset of first 1 bit

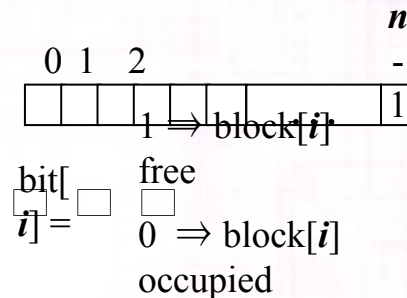
CPUs have instructions to return offset within word of first “1” bit





Free-Space Management

- File system maintains free-space list to track available blocks
- Bit vector or bit map (n blocks)



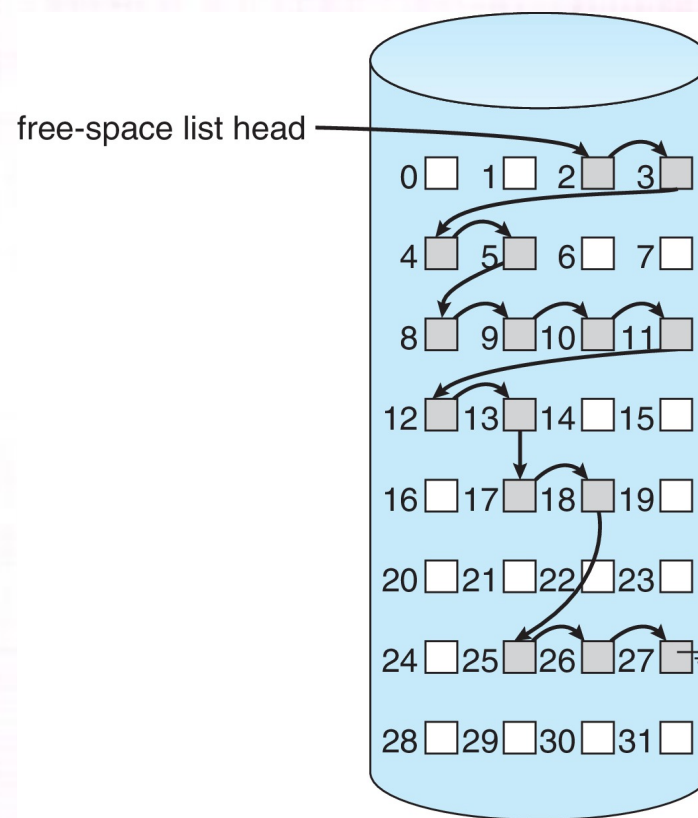
- Bit map requires extra space
 - Example:
 - block size = 4KB = 212 bytes
 - disk size = 240 bytes (1 terabyte)
 - $n = 240/212 = 228$ bits (or 32MB)
 - if clusters of 4 blocks $\rightarrow$ 8MB of memory
- Easy to get contiguous files





Linked Free Space List on Disk

- Linked list (free list)
 - Cannot get contiguous space easily
 - No waste. Linked Free Space List on Disk of space
 - No need to traverse the entire list (if # free blocks recorded)





Free-Space Management (Cont.)

- Grouping
 - Modify linked list to store address of next $n-1$ free blocks in first free block, plus a pointer to next block that contains free-block-pointers (like this one)
- Counting
 - Because space is frequently contiguously used and freed, with contiguous-allocation allocation, extents, or clustering
 - Keep address of first free block and count of following free blocks
 - Free space list then has entries containing addresses and counts



Free-Space Management (Cont.)

- Space Maps
 - Used in ZFS
 - Consider meta-data I/O on very large file systems
 - Full data structures like bit maps cannot fit in memory $\Rightarrow$ thousands of I/Os
 - Divides device space into metaslab units and manages metaslabs
 - Given volume can contain hundreds of metaslabs
 - Each metaslab has associated space map
 - Uses counting algorithm
 - But records to log file rather than file system
 - Log of all block activity, in time order, in counting format
 - Metaslab activity $\Rightarrow$ load space map into memory in balanced-tree structure, indexed by offset
 - Replay log into that structure
 - Combine contiguous free blocks into single entry





Efficiency and Performance

- Efficiency dependent on:
 - Disk allocation and directory algorithms
 - Types of data kept in file's directory entry
 - Pre-allocation or as-needed allocation of metadata structures
 - Fixed-size or varying-size data structures





Efficiency and Performance (Cont.)

- Performance
 - Keeping data and metadata close together
 - Buffer cache – separate section of main memory for frequently used blocks
 - Synchronous writes sometimes requested by apps or needed by OS
 - No buffering / caching – writes must hit disk before acknowledgement
 - Asynchronous writes more common, buffer-able, faster
 - Free-behind and read-ahead – techniques to optimize sequential access
 - Reads frequently slower than writes
 -





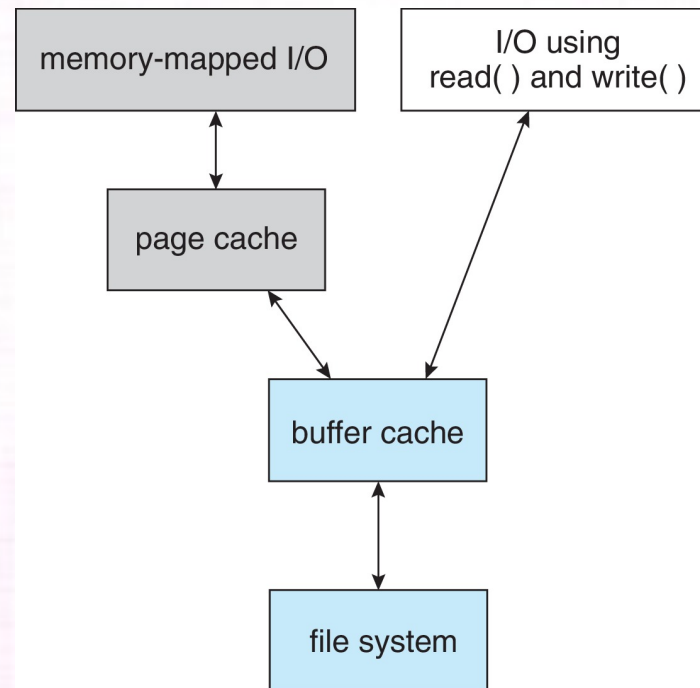
Page Cache

- A page cache caches pages rather than disk blocks using virtual memory techniques and addresses
- Memory-mapped I/O uses a page cache
- Routine I/O through the file system uses the buffer (disk) cache
- This leads to the following figure





I/O Without a Unified Buffer Cache





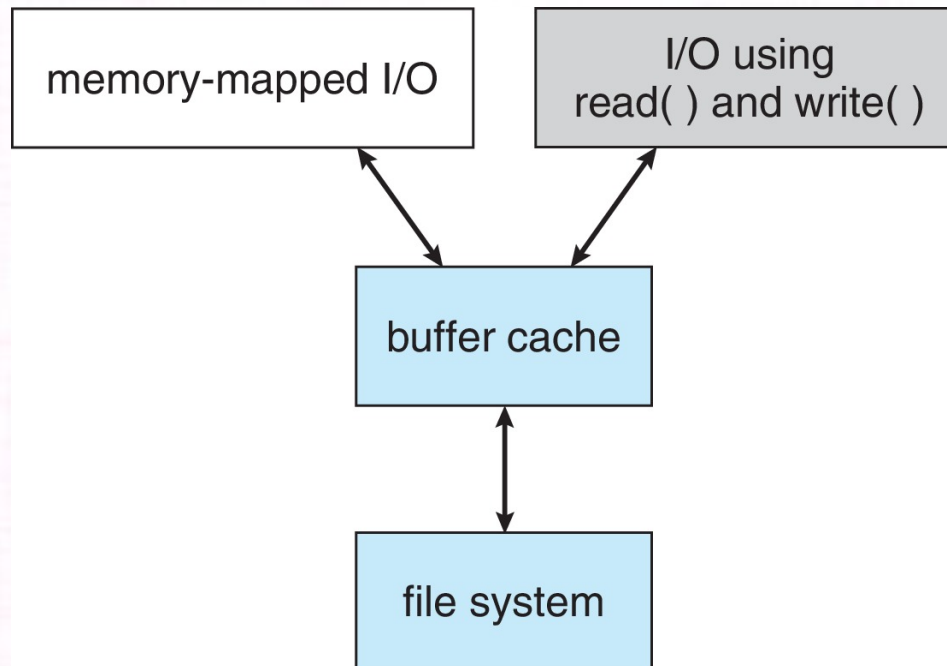
Unified Buffer Cache

- A unified buffer cache uses the same page cache to cache both memory-mapped pages and ordinary file system I/O to avoid double caching
 - But which caches get priority, and what replacement algorithms to use?





I/O Using a Unified Buffer Cache





Recovery

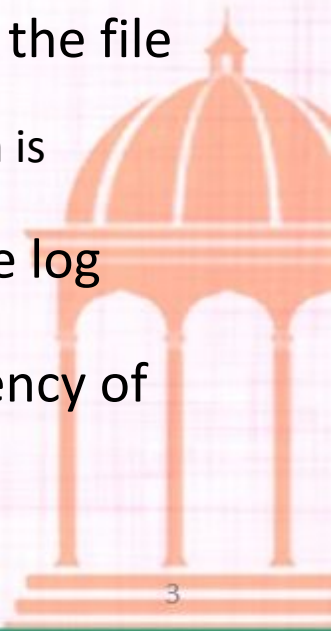
- Consistency checking – compares data in directory structure with data blocks on disk, and tries to fix inconsistencies
 - Can be slow and sometimes fails
- Use system programs to back up data from disk to another storage device (magnetic tape, other magnetic disk, optical)
- Recover lost file or disk by restoring data from backup





Log Structured File Systems

- Log structured (or journaling) file systems record each metadata update to the file system as a transaction
- All transactions are written to a log
 - A transaction is considered committed once it is written to the log (sequentially)
 - Sometimes to a separate device or section of disk
 - However, the file system may not yet be updated
- The transactions in the log are asynchronously written to the file system structures
 - When the file system structures are modified, the transaction is removed from the log
- If the file system crashes, all remaining transactions in the log must still be performed
- Faster recovery from crash, removes chance of inconsistency of metadata





Overview of Mass Storage Structure

- Bulk of secondary storage for modern computers is **hard disk drives (HDDs)** and **nonvolatile memory (NVM)** devices
- **HDDs** spin platters of magnetically-coated material under moving read-write heads
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time (random-access time)** is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface -- That's bad
- Disks can be removable





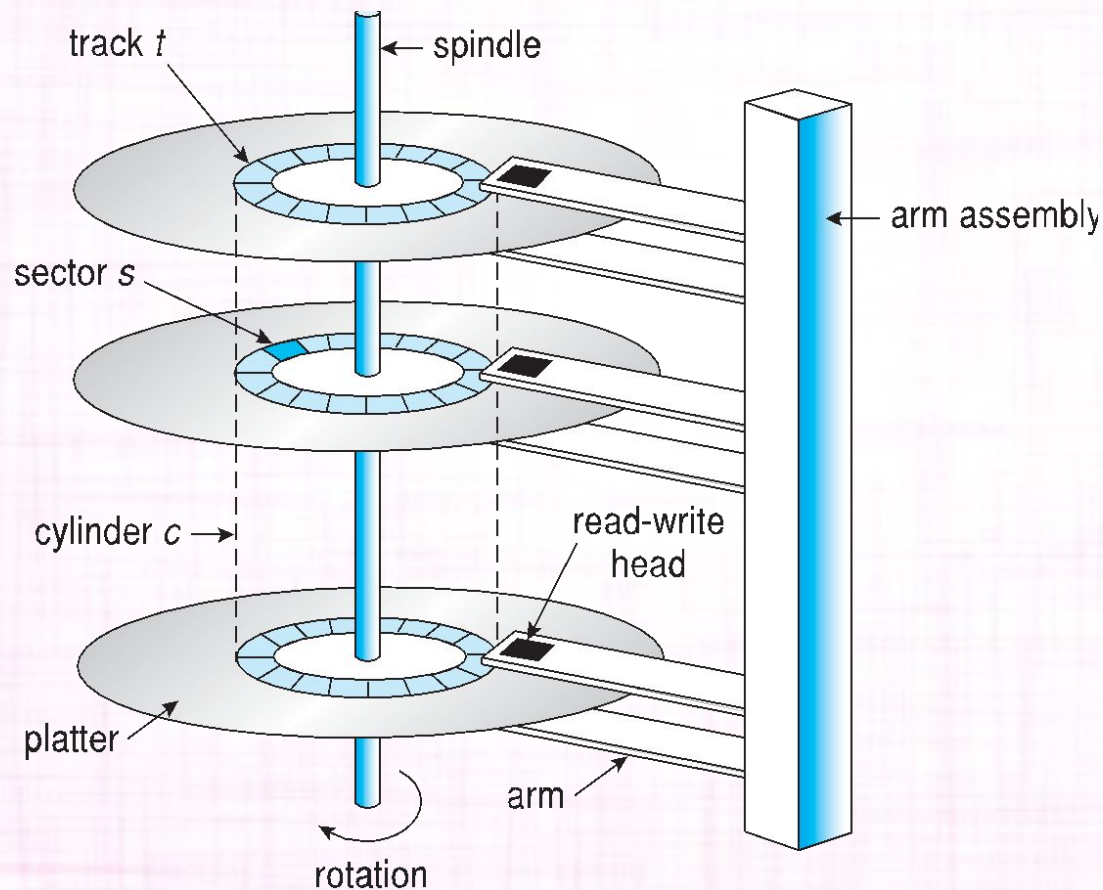
Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
 - Low-level formatting creates **logical blocks** on physical media
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
 - Sector 0 is the first sector of the first track on the outermost cylinder
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
 - Logical to physical address should be easy
 - Except for bad sectors
 - Non-constant # of sectors per track via constant angular velocity





Moving-head Disk Mechanism





Hard Disk Drives

- Platters range from .85" to 14" (historically)
 - Commonly 3.5", 2.5", and 1.8"
- Range from 30GB to 3TB per drive
- Performance
 - Transfer Rate – theoretical – 6 Gb/sec
 - Effective Transfer Rate – real – 1Gb/sec
 - Seek time from 3ms to 12ms – 9ms common for desktop drives
 - Average seek time measured or calculated based on 1/3 of tracks
 - Latency based on spindle speed
 - $1 / (\text{RPM} / 60) = 60 / \text{RPM}$
 - Average latency = $\frac{1}{2}$ latency





Hard Disk Performance

- **Access Latency = Average access time =**
average seek time + average latency
 - For fastest disk $3\text{ms} + 2\text{ms} = 5\text{ms}$
 - For slow disk $9\text{ms} + 5.56\text{ms} = 14.56\text{ms}$
- Average I/O time = average access time + (amount to transfer / transfer rate) + controller overhead
- For example to transfer a 4KB block on a 7200 RPM disk with a 5ms average seek time, 1Gb/sec transfer rate with a .1ms controller overhead =
 - $5\text{ms} + 4.17\text{ms} + 0.1\text{ms} + \text{transfer time} =$
 - $\text{Transfer time} = 4\text{KB} / 1\text{Gb/s} * 8\text{Gb} / \text{GB} * 1\text{GB} / 1024^2\text{KB}$
 $= 32 / (1024^2) = 0.031 \text{ ms}$
 - Average I/O time for 4KB block = $9.27\text{ms} + .031\text{ms} = 9.301\text{ms}$





The First Commercial Disk Drive



1956
IBM RAMDAC computer included the
IBM Model 350 disk storage system

5M (7 bit) characters
50 x 24" platters
Access time = < 1 second





Nonvolatile Memory Devices

- If disk-drive like, then called **solid-state disks (SSDs)**
- Other forms include **USB drives** (thumb drive, flash drive), DRAM disk replacements, surface-mounted on motherboards, and main storage in devices like smartphones
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span – need careful management
- Less capacity
- But much faster
- Busses can be too slow -> connect directly to PCI for example
- No moving parts, so no seek time or rotational latency





Nonvolatile Memory Devices

- Have characteristics that present challenges
- Read and written in “page” increments (think sector) but can’t overwrite in place
 - Must first be erased, and erases happen in larger “block” increments
 - Can only be erased a limited number of times before worn out – ~ 100,000
 - Life span measured in **drive writes per day (DWPD)**
 - A 1TB NAND drive with rating of 5DWPD is expected to have 5TB per day written within warranty period without failing





NAND Flash Controller Algorithms

- With no overwrite, pages end up with mix of valid and invalid data
- To track which logical blocks are valid, controller maintains **flash translation layer (FTL)** table
- Also implements **garbage collection** to free invalid page space
- Allocates **overprovisioning** to provide working space for GC
- Each cell has lifespan, so **wear leveling** needed to write equally to all cells

valid page	valid page	invalid page	invalid page
invalid page	valid page	invalid page	valid page

NAND block with valid and invalid pages





Volatile Memory

- DRAM frequently used as mass-storage device
 - Not technically secondary storage because volatile, but can have file systems, be used like very fast secondary storage
- **RAM drives** (with many names, including RAM disks) present as raw block devices, commonly file system formatted
- Computers have buffering, caching via RAM, so why RAM drives?
 - Caches / buffers allocated / managed by programmer, operating system, hardware
 - RAM drives under user control
 - Found in all major operating systems
 - Linux `/dev/ram`, macOS `diskutil` to create them, Linux `/tmp` of file system type `tmpfs`
- Used as high speed temporary storage
 - Programs could share bulk data, quickly, by reading/writing to RAM drive





Magnetic Tape

Magnetic tape was used as an early secondary-storage medium. Although it is nonvolatile and can hold large quantities of data, its access time is slow compared with that of main memory and drives. In addition, random access to magnetic tape is about a thousand times slower than random access to HDDs and about a hundred thousand times slower than random access to SSDs so tapes are not very useful for secondary storage. Tapes are used mainly for backup, for storage of infrequently used information, and as a medium for transferring information from one system to another.

A tape is kept in a spool and is wound or rewound past a read–write head. Moving to the correct spot on a tape can take minutes, but once positioned, tape drives can read and write data at speeds comparable to HDDs. Tape capacities vary greatly, depending on the particular kind of tape drive, with current capacities exceeding several terabytes. Some tapes have built-in compression that can more than double the effective storage. Tapes and their drivers are usually categorized by width, including 4, 8, and 19 millimeters and 1/4 and 1/2 inch. Some are named according to technology, such as LTO-6 (Figure 11.5) and SDLT.



Figure 11.5 An LTO-6 Tape drive with tape cartridge inserted.



Disk Attachment

- Host-attached storage accessed through I/O ports talking to **I/O busses**
- Several busses available, including **advanced technology attachment (ATA)**, **serial ATA (SATA)**, **eSATA**, **serial attached SCSI (SAS)**, **universal serial bus (USB)**, and **fibre channel (FC)**.
- Most common is SATA
- Because NVM much faster than HDD, new fast interface for NVM called **NVM express (NVMe)**, connecting directly to PCI bus
- Data transfers on a bus carried out by special electronic processors called **controllers** (or **host-bus adapters, HBAs**)
 - Host controller on the computer end of the bus, device controller on device end
 - Computer places command on host controller, using memory-mapped I/O ports
 - Host controller sends messages to device controller
 - Data transferred via DMA between device and computer DRAM





Address Mapping

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
 - Low-level formatting creates **logical blocks** on physical media
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
 - Sector 0 is the first sector of the first track on the outermost cylinder
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
 - Logical to physical address should be easy
 - Except for bad sectors
 - Non-constant # of sectors per track via constant angular velocity





HDD Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time $\approx$ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer





Disk Scheduling (Cont.)

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists
- In the past, operating system responsible for queue management, disk drive head scheduling
 - Now, built into the storage devices, controllers
 - Just provide LBAs, handle sorting of requests (**Logical Block Addressing.**)
 - Some of the algorithms they use described next



Disk Scheduling (Cont.)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

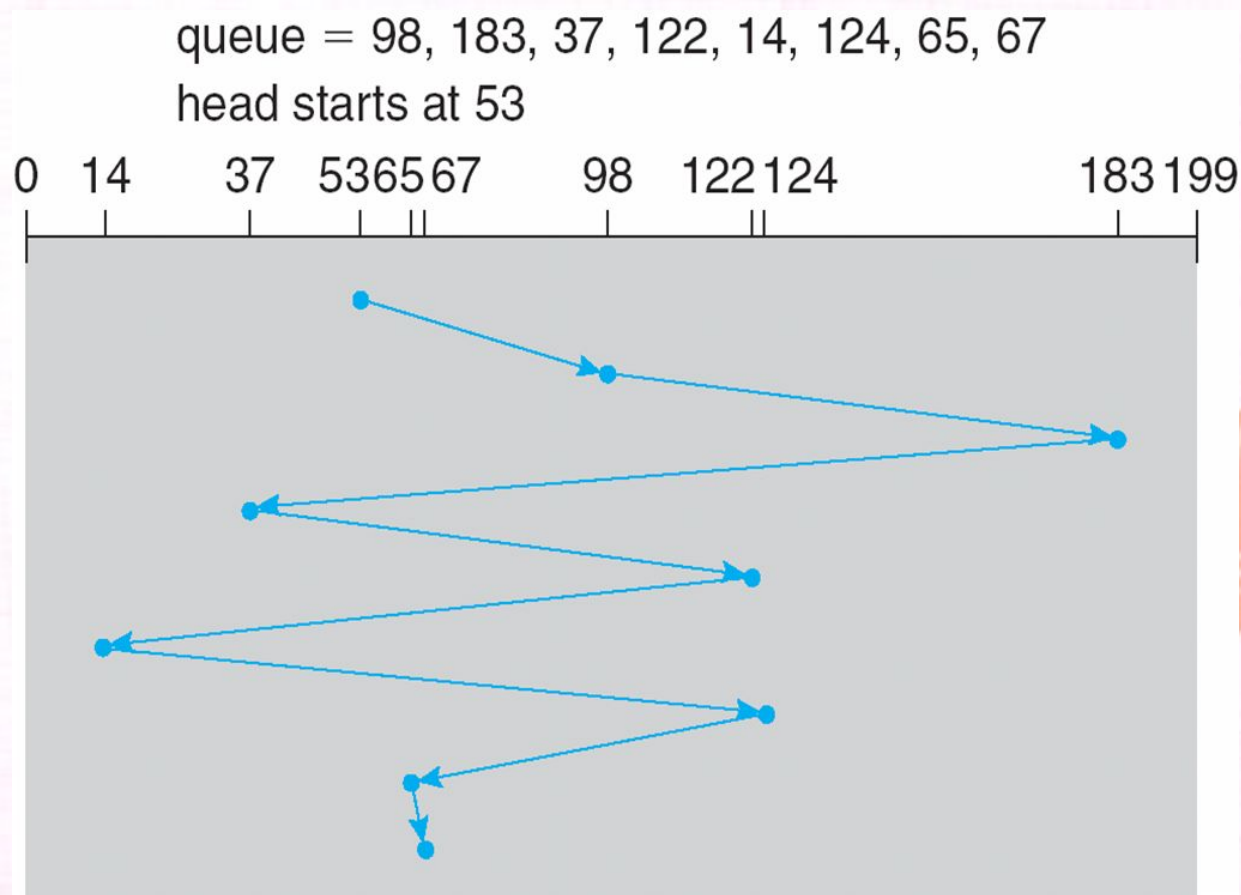
Head pointer 53





FCFS

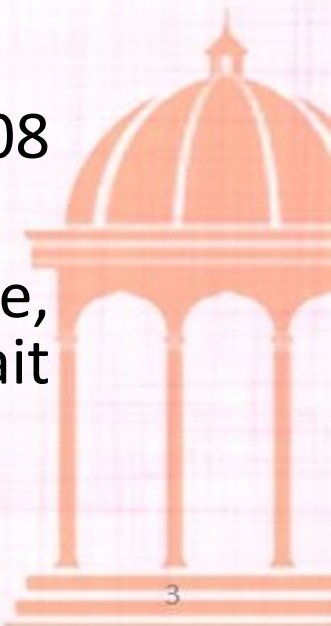
Illustration shows total head movement of 640 cylinders





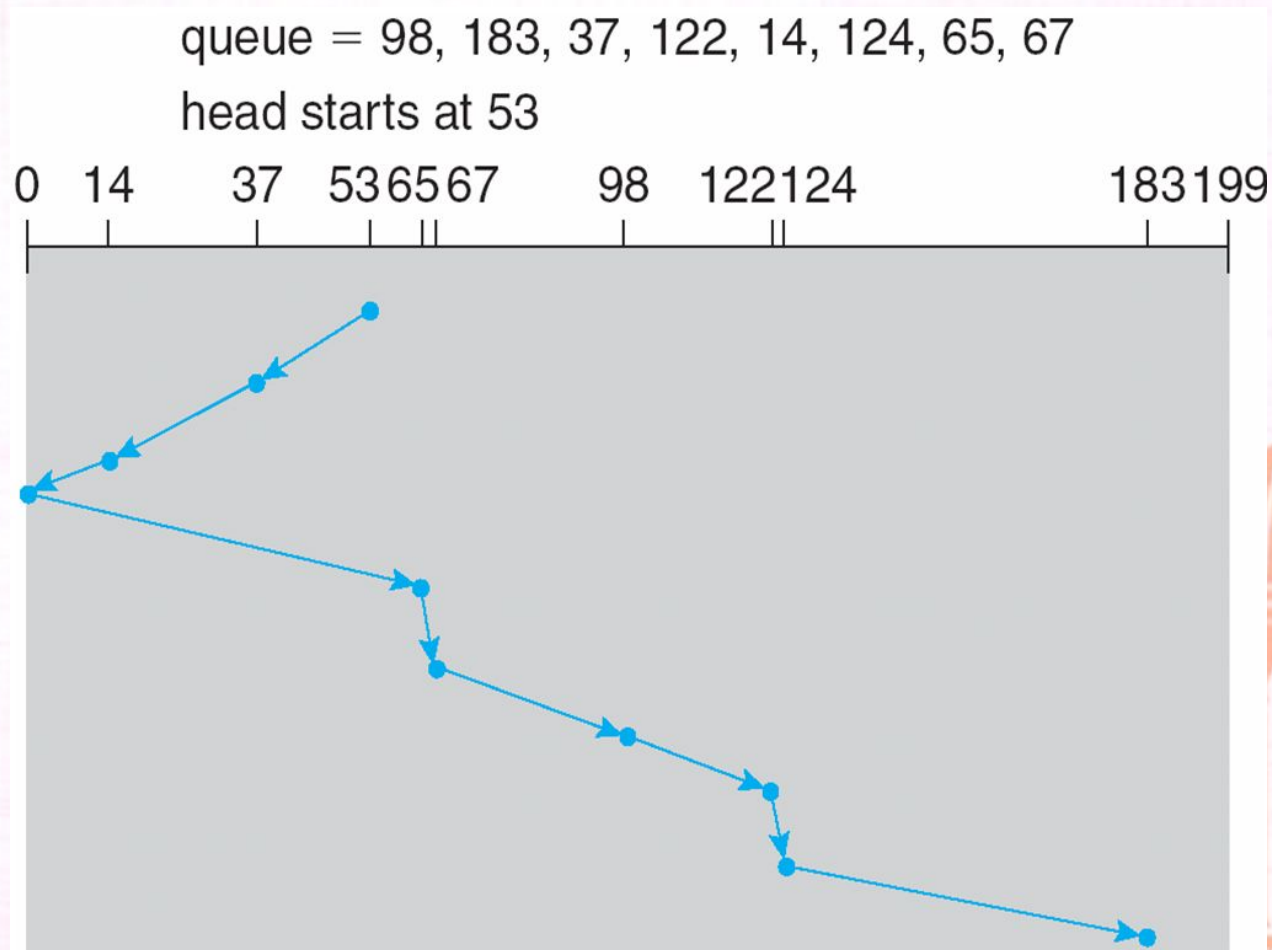
SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- Illustration shows total head movement of 208 cylinders
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest





SCAN (Cont.)





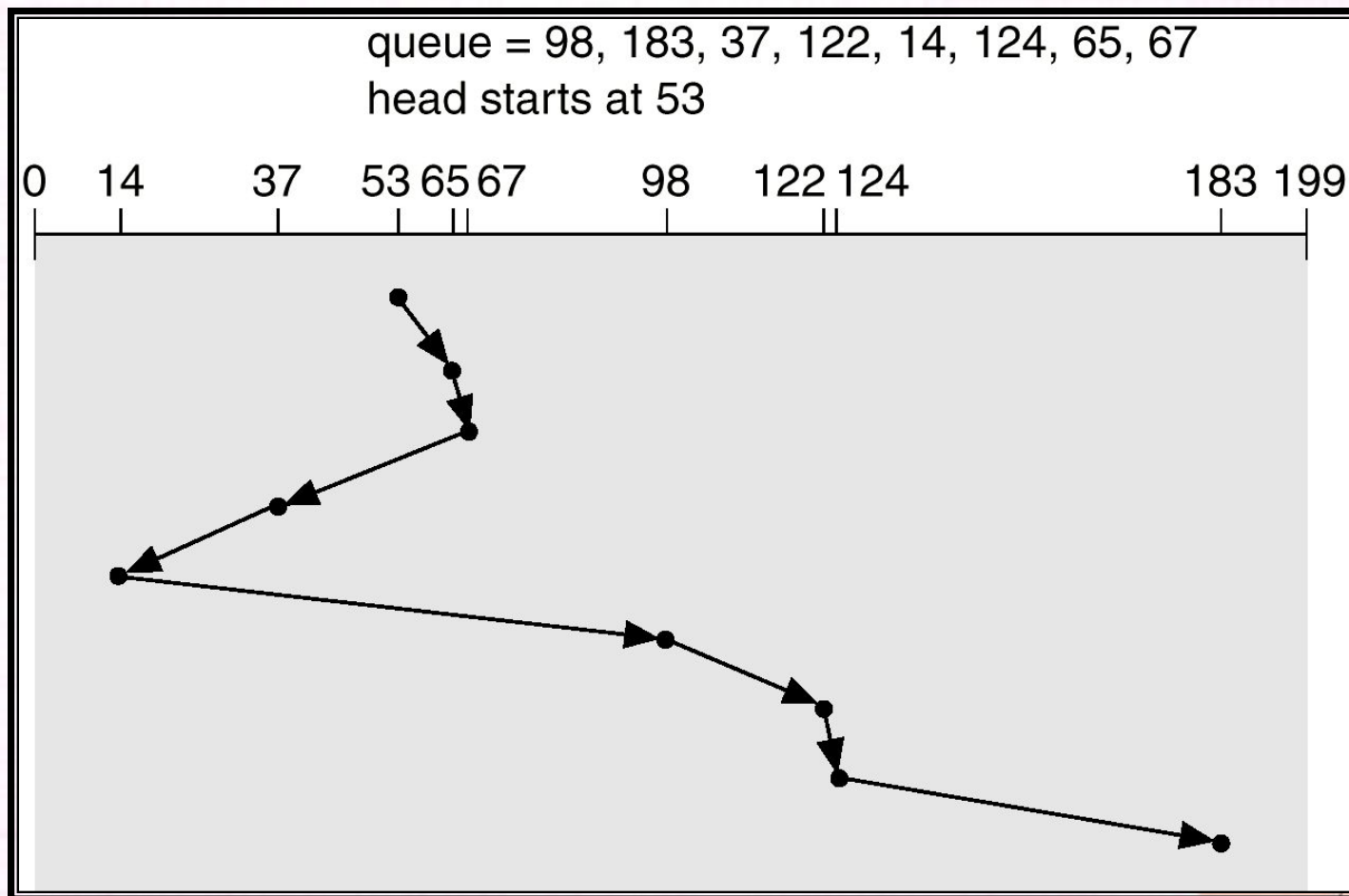
SSTF

- Selects the request with the minimum seek time from the current head position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.
- Illustration shows total head movement of 236 cylinders.





SSTF (Cont.)





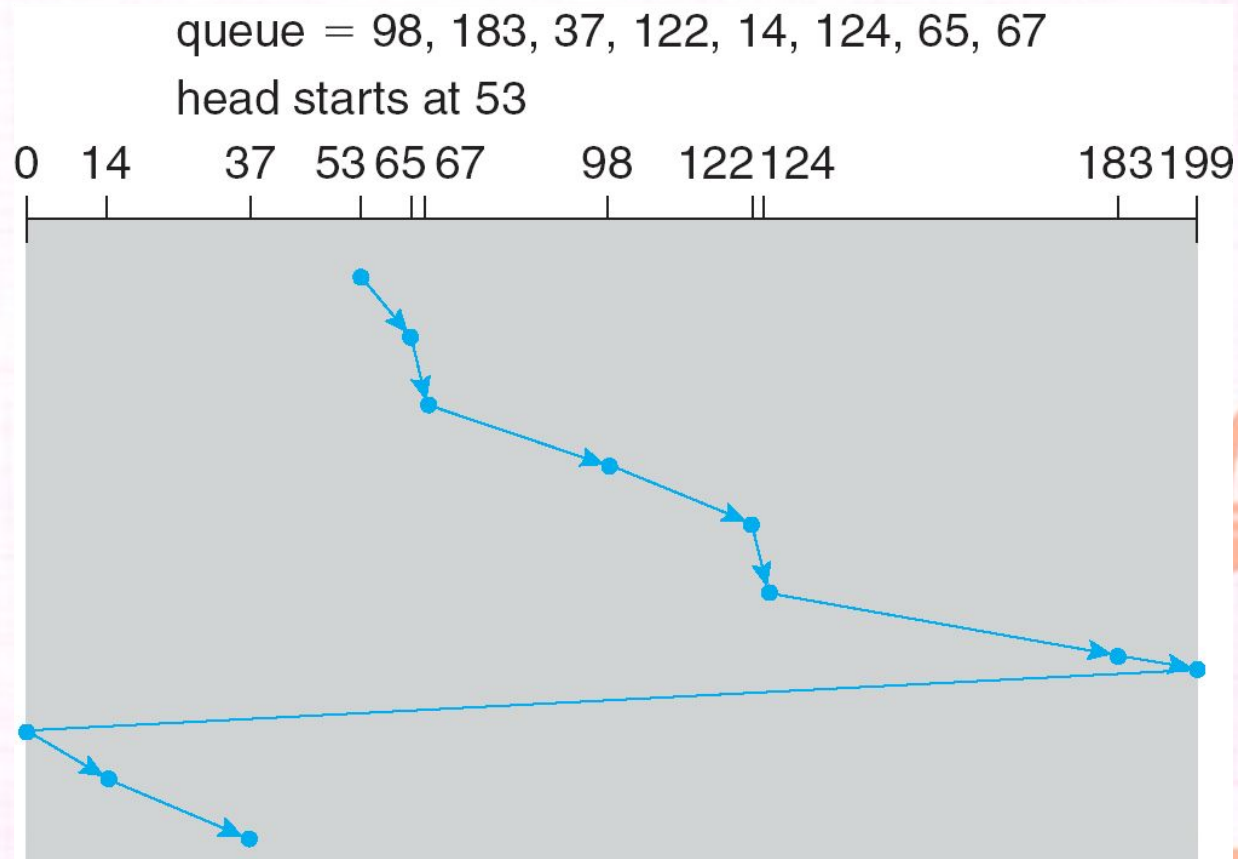
C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?





C-SCAN (Cont.)





Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation, but still possible
- To avoid starvation Linux implements **deadline** scheduler
 - Maintains separate read and write queues, gives read priority
 - Because processes more likely to block on read than write
 - Implements four queues: 2 x read and 2 x write
 - 1 read and 1 write queue sorted in LBA order, essentially implementing C-SCAN
 - 1 read and 1 write queue sorted in FCFS order
 - All I/O requests sent in batch sorted in that queue's order
 - After each batch, checks if any requests in FCFS older than configured age (default 500ms)
 - If so, LBA queue containing that request is selected for next batch of I/O
- In RHEL 7 also **NOOP** and **completely fair queueing** scheduler (**CFQ**) also available, defaults vary by storage device



NVM Scheduling

- No disk heads or rotational latency but still room for optimization
- In RHEL 7 **NOOP** (no scheduling) is used but adjacent LBA requests are combined
 - NVM best at random I/O, HDD at sequential
 - Throughput can be similar
 - **Input/Output operations per second (IOPS)** much higher with NVM (hundreds of thousands vs hundreds)
 - But **write amplification** (one write, causing garbage collection and many read/writes) can decrease the performance advantage



Swap-Space Management

MIT School of Computing
Department of Computer Science & Engineering



MIT-ADT
UNIVERSITY
PUNE, INDIA

- Used for moving entire processes (swapping), or pages (paging), from DRAM to secondary storage when DRAM not large enough for all processes
- Operating system provides **swap space management**
 - Secondary storage slower than DRAM, so important to optimize performance
 - Usually multiple swap spaces possible – decreasing I/O load on any given device
 - Best to have dedicated devices
 - Can be in raw partition or a file within a file system (for convenience of adding)
 - Data structures for swapping on Linux systems:

